

Program: Storing a Document with Embedded Subdocuments in MongoDB using Mongoose

```
const mongoose = require('mongoose');
```

- This imports the **Mongoose** library.
- Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It helps us define schemas and interact with the database more easily.

```
const { Schema, model } = mongoose;
```

- This extracts the `Schema` and `model` classes from `mongoose`.
- We use `Schema` to define the structure of our documents (like blueprints), and `model` to create models that we use to interact with collections in MongoDB.

```
const addressSchema = new Schema({
  street: String,
  city: String,
  country: String,
});
```

- Creates a **schema** for the embedded document called `addressSchema`.
- **Details:**
This schema defines how an address looks — it has 3 fields: `street`, `city`, and `country`, all of which are strings.
- Since users can have multiple addresses, we create this sub-schema to embed inside the user document.

```
const userSchema = new Schema({
  name: String,
  email: String,
  addresses: [addressSchema]
});
```

- Defines the schema for the main document, a **User**.
- **Details:**
The User document has:
 - o `name` (String)
 - o `email` (String)
 - o `addresses` which is an **array of embedded address documents** defined by `addressSchema`.
- This design models a user with multiple addresses embedded inside their document.

```
const User = model('User', userSchema);
```

- Creates a **model** called `User` based on `userSchema`.
- Models let you create and read documents from the corresponding MongoDB collection (`users` collection by default, pluralized lowercase of the model name).

```
async function run() {
```

- Declares an **asynchronous function** named `run`.
- MongoDB operations are asynchronous (they take time to complete). Using `async` allows us to use `await` inside to wait for these operations.

```
await mongoose.connect('mongodb://localhost:27017/testdb');
```

- Connects to a MongoDB database named `testdb` running locally on your machine.
- Before doing anything with the database, you must establish a connection.

```
const user = new User({
  name: 'John Doe',
  email: 'john@example.com',
  addresses: [
    { street: '123 Main St', city: 'New York', country: 'USA' },
    { street: '456 Elm St', city: 'Boston', country: 'USA' }
  ]
});
```

- Creates a new **User document instance** with the provided data.
- **Details:**
 - Name and email are strings.
 - `addresses` is an array of two address objects, each matching the `addressSchema`.
- This shows how you create a user with embedded addresses before saving it to the database.

```
await user.save();
```

- Saves the newly created user document into the `users` collection in MongoDB.
- Until this line runs, the user only exists in your program's memory, not in the database.

```
console.log('User saved successfully!');
```

- Prints a confirmation message to the console after the save operation completes.
- To notify you that the user document was saved without errors.

```
await mongoose.disconnect();
```

- Closes the connection to the MongoDB database.
- It's a good practice to close the database connection when you are done to free up resources.

```
run();
```

- Calls the `run` function to execute the above asynchronous logic.
 - This triggers the database connection, creation, saving, and disconnecting flow.
-

Summary

- You import and use Mongoose to define data structures (schemas).
 - Define a **sub-schema** for addresses, then embed those inside the main user schema.
 - Create a user object with multiple addresses and save it to MongoDB.
 - Always connect before, and disconnect after, database operations.
-

✓ Step-by-Step Execution Guide (VS Code + MongoDB)

🎯 Step 1: Make Sure MongoDB Server is Running

1. **Open a new terminal (Command Prompt or VS Code terminal).**
2. Run:

```
mongod --dbpath "C:\data\db"
```

👉 **Keep this window open.** This is your **MongoDB server** window.

✓ Expected: It should say **"Waiting for connections on port 27017"**

🎯 Step 2: Open your project folder in VS Code

- Your file should be something like `userWithAddresses.js` (you can name anything).
 - Make sure your file is saved.
-

🎯 Step 3: Install Mongoose (one time setup)

In **another new terminal** inside VS Code, run:

```
npm init -y  
npm install mongoose
```

✓ This creates `package.json` and installs **Mongoose**.

Step 4: Run Your Code

Still in VS Code terminal (second terminal, NOT the one with `mongod`), run:

```
node userWithAddresses.js
```

(Use the **actual filename** you saved)

👉 Expected output:

```
User saved successfully!
```

Step 5: (Optional) Verify Data Was Inserted

Open **another terminal** or **Mongo Shell** and type:

```
mongo
use testdb
db.users.find().pretty()
```

You should see John Doe with **embedded addresses** 